LECTURES

12 Event basics | 20 Event object | 21 All events | 18 Forms | 13 Date | ★ Cheat sheet

REVISION NOTES · JAVASCRIPT

JavaScript Events and Forms Notes

Condensed from five lecture decks (Event fire, Date function, Form, Event Object, All Event). Each section ends with the points most likely to be asked. Code is cleaned up from the slides: curly quotes and small typos are fixed.

Events: how to attach and fire them

Deck 12 · event_fire

An event is something the browser does or the user does: the page finished loading, an input changed, a button was clicked. JavaScript runs code when it detects the event.

Three ways to attach a handler

WAY	EXAMPLE	NOTES
Inline attribute	`onclick="go()"`	Handler written in the HTML tag. Can hold code directly: `onclick="alert('hi')"`
Property	`btn.onclick = go`	Pass the function name **without** `()`. Only one handler per event per element; a second assignment replaces the first.
`addEventListener`	`btn.addEventListener("click", go)`	Event name has no `on` prefix. Many handlers per event. Can be removed again.

```
<!-- 1. inline -->
<input type="button" value="Click" onclick="go()">
<input type="button" value="Click" onclick="alert('button == clicked')">

<!-- 2. property -->
```

```
<input type="button" id="btn" value="Click">
<script>
  function go(){ alert("click event"); }
  document.getElementById("btn").onclick = go;   // no ()
</script>
```

addEventListener and removeEventListener

- `element.addEventListener(event, function)` attaches a handler to that element.
- `document.addEventListener()` attaches one to the whole document.
- `removeEventListener()` removes a handler added with `addEventListener`.
- Not supported in Internet Explorer 8 and earlier.

```
document.getElementById("btn").addEventListener("click", go);

document.getElementById("pp").addEventListener("mouseover", function rise(){
  alert("go rise");
});
```

Removing a handler

```
function myFunction(){ document.getElementById("demo").innerHTML = Math.random(); }
function removeHandler(){
  document.getElementById("myDIV").removeEventListener("mousemove", myFunction);
}
const box = document.getElementById("myDIV");
box.addEventListener("mousemove", myFunction);
box.addEventListener("mouseover", function(){ alert("hi") });
box.addEventListener("mouseout", () => alert("mouse out"));   // arrow function
```

WATCH OUT

`removeEventListener` only works if you pass the **SAME NAMED FUNCTION** that you added. An anonymous function or arrow function cannot be removed, because you have no reference to it.

Property style: `btn.onclick = go` (no brackets). With brackets, `go()` runs immediately and its return value gets assigned.

The event object

Deck 20 : Event Object

The browser passes an `event` object to every handler. `Event` is the parent of all event objects. Use `onclick="myFunction(event)"` to pass it in an inline handler, or accept it as the first parameter of a function added with `addEventListener`.

```
function whichType(){
  alert(event.type);      // "keydown"
  console.log(event);     // inspect the whole object
}
// <body onkeydown="whichType()">
```

screenX/screenY vs clientX/clientY

PROPERTY	MEASURED FROM
`event.screenX`, `event.screenY`	Top-left of your **desktop screen**
`event.clientX`, `event.clientY`	Top-left of the **browser viewport** (0,0 is the page's visible corner)

```
function coordinates(){
  alert("screen: " + event.screenX + ", " + event.screenY);
  alert("client: " + event.clientX + ", " + event.clientY);
}
// <body onmousedown="coordinates()">
```

event.target vs event.currentTarget

	EVENT.TARGET	EVENT.CURRENTTARGET
Returns	The element that **triggered** the event (what you actually clicked)	The element whose **listener** is running
Example: `<body onclick>`, you click a `<p>`	`P`	`BODY`
Useful for	Finding what was clicked inside a container	Capturing and bubbling, where listener and origin differ

```
// target
```

```
function myFunction(event){
  var x = event.target;
  document.getElementById("demo").innerHTML = "Triggered by a " + x.nodeName + " eleme
}
// click h1 -> "Triggered by a H1 element"; click button -> "BUTTON"

// currentTarget
function myFunction(event){ alert(event.currentTarget.nodeName); }  // always BODY her
```

REMEMBER

`target` = who started it. `currentTarget` = who is listening. They are the same only when you click the element that owns the listener.

Common events

Deck 21 · AllEvent

EVENT	FIRES WHEN	TYPICAL USE
`onfocus`	A component gets focus (you click into it)	Highlight the field, give it a border
`onblur`	A component loses focus	Validate the field after the user leaves it
`onmousemove`	The mouse moves over the element	Tracking pointer movement
`onerror`	A document or image fails to load	Fallback or message for a missing image
`onchange`	A value is committed (dropdown choice changes; text field loses focus after edit)	Read a selected option
`oninput`	Every time the value is modified	Live preview of what is typed
`onkeydown` / `onkeypress` / `onkeyup`	Key pressed / character key pressed / key released	Filtering typing, shortcuts
`onmouseleave` / `onmouseout`	Pointer leaves the element	Hover effects

onfocus and onblur

With two text boxes t1 and t2: click t1, then move to t2. t1 gets `onblur`, t2 gets `onfocus`. Field validation is usually done on blur.

```
First name: <input type="text" id="fname" onblur="change()">
Last name:  <input type="text" id="lname" onfocus="setStyle()">
<script>
  function setStyle(){ document.getElementById("lname").style.background = "yellow"; }
  function change(){   document.getElementById("fname").style.color = "blue"; }
</script>
```

onerror

```
<img src="imae.gif" onerror="alert('The image could not be loaded.')">
<!-- the file does not exist, so the browser fires onerror -->
```

onchange with a dropdown

```
function favBrowser(){
  var list = document.getElementById("myList");
  alert(list.selectedIndex);
  document.getElementById("favorite").value = list.options[list.selectedIndex].text;
}
// <select id="myList" onchange="favBrowser()"> ... </select>
```

oninput

- Fires after every modification by the user, including pasting with the mouse and speech dictation, which keyboard events miss.
- Does **not** fire for keys that change nothing, such as the arrow keys.
- Fires **after** the value changed, so `event.preventDefault()` has no effect. You cannot block input with it.

```
document.getElementById("input").oninput = function(){
  document.getElementById("result").innerHTML = document.getElementById("input").value
};
```

Keyboard events

Order: `keydown` then `keypress` then `keyup`.

```
EVENT       RAISED                                  NON-CHARACTER KEYS (ARROWS,
```

		SHIFT...)?
keydown	As soon as the key goes down, while still held	Yes
keypress	For character keys while pressed. Higher-level event	No
keyup	After the key is released	Yes

The slides recommend handling `keyup`: start actions only after release, and it has full key information, including non-character keys.

```
function noNumbers(e){
  var keynum  = e.keyCode;
  var keychar = String.fromCharCode(keynum);
  var numcheck = /\d/;
  return !numcheck.test(keychar);     // false blocks the key
}
// <input type="text" onkeypress="return noNumbers(event)">   digits are blocked
// <input type="text" onkeydown="return false">               nothing can be typed
```

RETURN VALUE RULE

Every event handler returns `true` by default. Writing `onkeydown="return false"` cancels the default action, so the character never appears.

onmouseleave vs onmouseout

- `onmouseleave` fires only when the pointer leaves the div itself.
- `onmouseout` fires when the pointer leaves the div **and also** when it moves out of any child (the `<p>` or `<span>` inside it).
- In the slide demo, the counter for `onmouseout` goes up faster than the one for `onmouseleave` as you move around inside the box.

WATCH OUT

`onchange` on a text box fires on blur, not on every keystroke. For live updates use `oninput`.

In current browsers `keypress` and `keyCode` are deprecated; `keydown` with `event.key` is the modern way. The slides use the older names, so use those in answers for this course.

Forms and the DOM

Deck 18 · Form

The page is a set of arrays

Everything in the page is held in arrays. `document.forms[]` holds each form in order, and each form has an `elements[]` array holding its controls in order.

```
document.forms.length            // number of forms on the page
document.forms[0].elements.length // controls inside the first form
document.forms[0].elements[i].value

// Two forms, each: radio, radio, button
// forms[0].elements[0], [1], [2]
// forms[1].elements[0], [1], [2]
```

The DOM hierarchy from the slides: `window` contains `history`, `document`, `location`, `screen`, `navigator`, `frames`, `event`. `document` contains `all`, `children`, `selection`, `forms`, `body`, `links`, and form controls such as text, button, radio, textarea, select, password, file, checkbox, submit, reset, option.

Access a field by name

```
<form name="frm">
  name <input name="t1">
  <input type="button" value="copy" onclick="call()">
  copied name <input name="t2">
</form>
<script>
  function call(){
    var a = document.frm.t1;     // form name, then field name
    var b = document.frm.t2;
    b.value = a.value;           // .value reads or writes the text
  }
</script>
```

Pattern: `document.formName.fieldName` returns the element object; `.value` gives its content.

Pass the form with `this.form`

Inside a control's handler, `this` is the control and `this.form` is the form that contains it.

```
function call(a){                // a = the form
  var v = a.length;              // number of elements
  var data = "";
```

```
  var data = "";
  for (var i = 0; i < v; i++) data += a.elements[i].value;
  document.getElementById("pp").innerHTML = data;
}
// <input type="button" value="click" onclick="call(this.form)">
```

Reading each control type

CONTROL	READ WITH	RETURNS
Text box	`document.frm.t.value`	String
Radio	`document.frm.rd[0].checked`	Boolean. Radios sharing one `name` form an array
Checkbox	`document.frm.a.checked`	Boolean. Check each box separately
Dropdown	`sl.selectedIndex` , `sl[i].selected` , `sl[i].text` , `sl[i].value`	Index, Boolean, label, value

Radio button

```
function readradio(){
  var p = document.getElementById("pp");
  if (document.frm.rd[0].checked) p.innerHTML = document.frm.rd[0].value;
  else                            p.innerHTML = document.frm.rd[1].value;
}
function cc(){ document.frm.rd[0].checked = true; }   // select it from code
// <input type="radio" name="rd" value="yes">yes
// <input type="radio" name="rd" value="no" checked>no     (checked = default choice)
```

Checkbox

```
function readchk(){
  var s = "";
  if (document.frm.a.checked) s += document.frm.a.value;
  if (document.frm.b.checked) s += document.frm.b.value;
  if (document.frm.c.checked) s += document.frm.c.value;
  document.getElementById("dd").innerHTML = s;
}
```

Dropdown (select)

```
function readsel(){
  var l = document.frm.sl.length;
  for (var i = 0; i < l; i++){
    if (document.frm.sl[i].selected) document.frm.t.value = document.frm.sl[i].value;
  }
  var index = document.frm.sl.selectedIndex;
  document.getElementById("pp").innerHTML = index + document.frm.sl[index].text;
}
// <select name="sl" onchange="readsel()">
//   <option value="0">dance</option> ...
```

`text` is what the user sees between the option tags; `value` is the value attribute.

Create a form dynamically

Pattern: `createElement`, set properties, `appendChild` into a parent, then append the finished form to a container in the page.

```
var f = document.createElement("form");
f.name = "frm";

var t = document.createElement("input");
t.type = "text"; t.name = "t1";

var r1 = document.createElement("input"); r1.type = "radio"; r1.name = "nm";
var l1 = document.createElement("label");
l1.appendChild(document.createTextNode("yes"));      // label text

var s = document.createElement("select");
var o = document.createElement("option"); o.innerHTML = "India";
s.appendChild(o);

f.appendChild(t); f.appendChild(r1); f.appendChild(l1); f.appendChild(s);
document.getElementById("dd").appendChild(f);        // show it
```

Form validation with `onsubmit`

`onsubmit` is a form-level event, fired when the submit button is clicked. By default it returns `true` and the form goes to its `action` page. Return `false` to stop submission.

```
<form name="frm" action="blur.html" onsubmit="return check()">
  <label>Password</label>
  <input type="text" name="nm">
  <span id="ter" class="err"></span>
  <input type="submit" value="submit">
</form>

function check(){
```

```
function check(){
  var len = document.frm.nm.value.length;
  var msg = document.getElementById("ter");
  if (len == 0){
    msg.innerHTML = "password can not be blank";
    document.frm.nm.focus();
    return false;
  } else if (len < 8 || len > 16){
    msg.innerHTML = "minimum 8 character required and can not exceed 15";
    return false;
  } else msg.innerHTML = "";          // valid: returns undefined, treated as true
}
function setcur(){ document.frm.nm.focus(); }   // autofocus, called by <body onload="
```

- The `return` in `onsubmit="return check()"` is required. Without it the false is ignored and the form still submits.
- If the function returns nothing for valid data, that is read as true and the browser moves to the action page.
- `focus()` puts the cursor in a field.

Make a plain button submit: `form.submit()`

A normal `type="button"` does not submit. Validate in its click handler and call `document.frm.submit()` yourself when the data is good.

```
// <input type="button" value="dum" onclick="check()">
function check(){
  var bool = true;
  var len = document.frm.nm.value.length;
  if (len == 0 || len < 8 || len > 16){ /* show message */ bool = false; }
  if (bool == true) document.frm.submit();     // fire the submit from code
}
```

WATCH OUT

`form.submit()` sends the form but does **NOT** run the `onsubmit` handler. That is why the validation sits in the button's own handler.

Validation with `isNaN()` and number conversion

```
isNaN("56 ket")   // true   not a number
isNaN(5)          // false
isNaN("5")        // false  numeric string counts as a number (value is converted)
isNaN(5 + 5)      // false
```

- Text box values are always **strings**. `"5" + "3"` gives `"53"`.
- Put a unary `+` in front to convert: `+n1.value + +n2.value` gives a real sum.
- `*` converts automatically, so `n1.value * n2.value` works without `+`.
- Check blank and non-numeric separately: `a.value.length == 0` and `isNaN(a.value)`.

Practice tasks from the deck

1. Three text boxes and two radios (`+` , `*`). Radio click shows the result in the third box, with validation.
2. Three checkboxes with book name and price. Show the total of the selected books in a disabled text box.
3. Dropdown of states. On change, show the selected state in a text box.
4. Three text boxes and a dropdown of math operations. A button shows the result in the third box.
5. Banner with 4 images that rotate every 1 second.
6. Personal details form with validation (rules below).
7. Digital clock.

```
// Q1 answer pattern
function dotask(op){
  let n1 = document.getElementById("n1"), n2 = document.getElementById("n2"), n3 = doc
  if (op == "+") n3.value = +n1.value + +n2.value;
  else if (op == "*") n3.value = n1.value * n2.value;
}
function validate(a, sid){
  let s = document.getElementById(sid);
  if (a.value.length == 0) s.innerHTML = "Text field can not be blank";
  if (isNaN(a.value))      s.innerHTML = "only number value allowed";
}
// <input id="n1" onchange="validate(this,'err1')"><span id="err1"></span>
// <input type="radio" name="r1" value="+" onchange="dotask(this.value)">

// Q2 answer pattern: add up the values of the checked boxes
function dosum(){
  var total = 0;
  if (document.frm.a.checked) total += +document.frm.a.value;   // C 250
  if (document.frm.b.checked) total += +document.frm.b.value;   // C# 750
  if (document.frm.c.checked) total += +document.frm.c.value;   // JavaScript 550
  document.getElementById("dd").innerHTML = total;
}
```

Q6 validation rules (registration form)

FIELD	RULE
Name	Minimum 5, maximum 15 characters
Password	Characters, numbers and symbols; minimum 8, maximum 15
Email	Contains `@` ; after the `.` there must be 2 or 3 characters
Address	Minimum 10, maximum 35 characters
DOB	Calendar input; age at least 18 and at most 80
Mobile	10 digits
Education	Dropdown
Gender	Male / Female radio buttons

The Date object

`Deck 13 · Date_function`

By default a date prints as a full text string in the browser's time zone, for example `Mon Nov 30 2020 13:03:21 GMT+0530 (India Standard Time)` . JavaScript stores a date as the number of **milliseconds since 1 January 1970, 00:00:00 UTC**.

Four ways to create a date

```
new Date()                                    // now
new Date(year, month, day, hours, minutes, seconds, milliseconds)
new Date(0)                                   // milliseconds since 1 Jan 1970 UTC
new Date("2015-03-25")                        // from a date string

var d = new Date(2018, 11, 24, 10, 33, 30, 0);   // Mon Dec 24 2018 10:33:30 (month 11
var d = new Date(2018, 11, 24, 10, 33, 30);      // milliseconds optional
```

String formats

TYPE	EXAMPLE
ISO date (preferred, `YYYY-MM-DD`)	`"2015-03-25"`
Short date	`"03/25/2015"`

Short date	`03/25/2015`
Long date	`"Mar 25 2015"` or `"25 Mar 2015"`

Parsing

```
var d = new Date("2015-03-25");
var msec = Date.parse("March 21, 2012");   // number of ms since 1 Jan 1970
var d2 = new Date(msec);                   // back to a Date
```

WATCH OUT

The result of `new Date("2015-03-25")` is computed relative to your time zone. Depending on the zone, the displayed day can be March 24 or March 25.

Get methods

METHOD	RANGE
`getFullYear()`	4-digit year
`getMonth()`	**0–11**
`getDate()`	1–31 (day of month)
`getDay()`	0–6 (weekday, 0 = Sunday)
`getHours()`	0–23
`getMinutes()`	0–59
`getSeconds()`	0–59
`getMilliseconds()`	0–999
`getTime()`	ms since 1 Jan 1970
`Date.now()`	Current time in ms (ECMAScript 5)

SET METHOD	SETS
`setFullYear(y, m, d)`	Year, optionally month and day
`setMonth()`	0–11

`setDate()`	1–31
`setHours()`	0–23
`setMinutes()`	0–59
`setSeconds()`	0–59
`setMilliseconds()`	0–999
`setTime()`	ms since 1 Jan 1970

```
var d = new Date();
d.setFullYear(2020);
d.setFullYear(2020, 11, 3);   // 3 Dec 2020
d.setMonth(11);               // December
d.setDate(15);
```

SAMPLE OUTPUT FROM THE SLIDE

For 30 Nov 2020: `getDate()` 30, `getDay()` 1 (Monday), `getMonth()` **10** (November), `getFullYear()` 2020, `getTime()` 1606729766545.

Digital clock

```
function startTime(){
  var today = new Date();
  var h = checkTime(today.getHours());
  var m = checkTime(today.getMinutes());
  var s = checkTime(today.getSeconds());
  document.getElementById("txt").innerHTML = h + ":" + m + ":" + s;
  setTimeout("startTime()", 500);        // run again after 500 ms
}
function checkTime(i){ if (i < 10) i = "0" + i; return i; }   // 7 -> "07"
// <body onload="startTime()"><div id="txt"></div>
```

Two ideas to remember: `checkTime` adds a leading zero to numbers below 10, and `setTimeout` calling its own function again keeps the clock ticking. The same `setTimeout` idea drives Q5, the image banner that rotates every second.

Last-minute cheat sheet

`All five decks`

Likely exam questions

- Three ways to attach an event handler, and what `addEventListener` adds over the other two.
- `target` vs `currentTarget`; `screenX` vs `clientX`.
- `onblur` vs `onfocus`; `onmouseleave` vs `onmouseout`; `oninput` vs `onchange`.
- Order of key events and which fire for non-character keys.
- How to read a text box, radio, checkbox, dropdown.
- How `onsubmit="return check()"` stops a submit; why `submit()` skips `onsubmit`.
- Why `getMonth()` starts at 0; what the number inside a Date is.

Easy mistakes

- Handler assigned as `go()` instead of `go`.
- Forgetting `return` in `onsubmit="return check()"`.
- Adding text box values without converting: `"2" + "3"` is `"23"`. Use `+` or `Number()`.
- Month counted from 1. January is 0, December is 11.
- Using `.value` on a radio group instead of looping over the array and checking `.checked`.
- Trying to remove an anonymous function with `removeEventListener`.
- Calling `preventDefault()` inside `oninput`. The value has already changed.
- Using the same `id` twice (the form slides repeat `id="n"`), which makes `getElementById` return only the first.